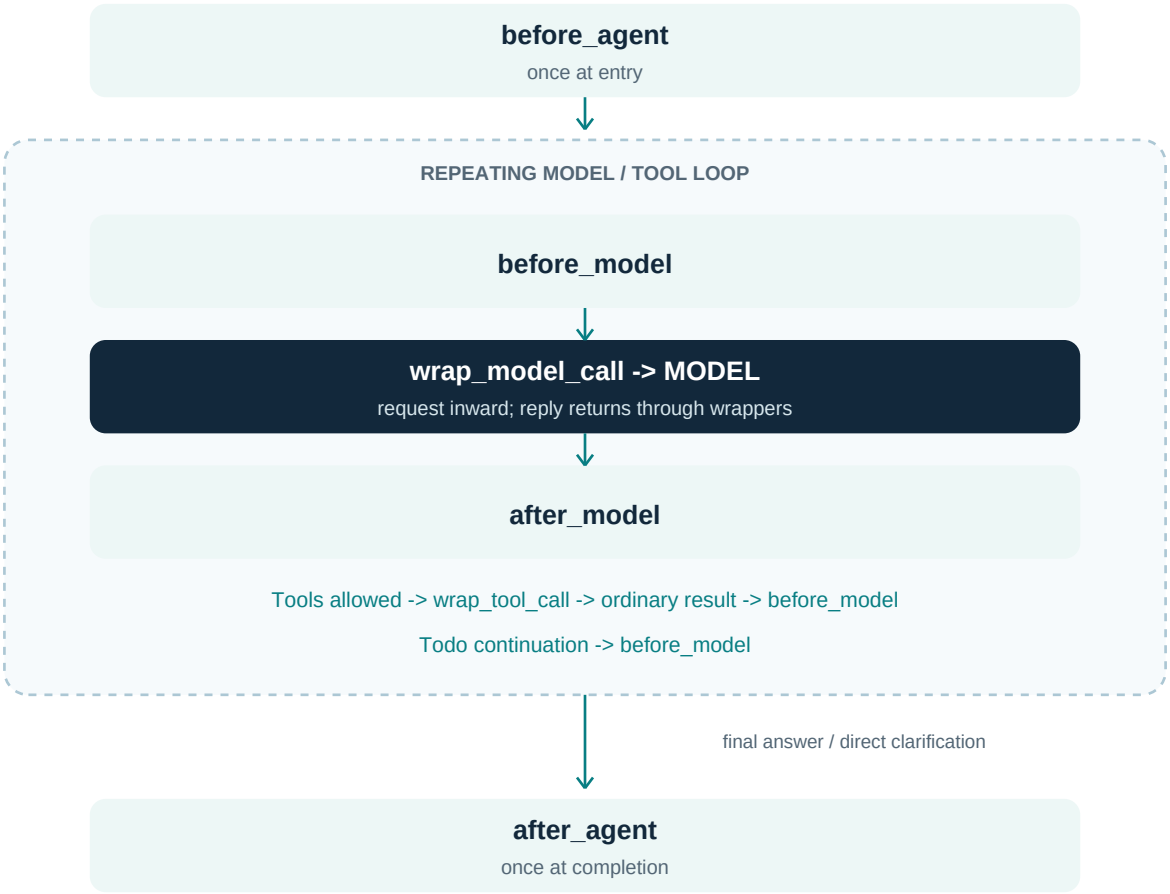
01 / THE BIG PICTURE

One turn. Several model calls.

The agent is the whole loop that handles your request. The model is one call inside that loop. Middleware adds preparation, checks and cleanup at specific points.



The rule to remember

before_agent and after_agent surround the complete invocation. before_model, model wrappers and after_model repeat as the agent works. A tool result normally goes back to before_model.

This companion explains the simplified DearFlow project with all 14 lead-agent middleware enabled. It refers to the existing 62-page dearflow-middleware-guide.pdf. Page numbers below refer to that original guide, not this companion.

Original guide: p. 3, p. 4, p. 6, p. 7

Three different things can change.

A changed prompt, a changed conversation state and a changed trace label are different outcomes. Knowing which one you are viewing prevents misleading conclusions.

Surface	Plain meaning	Typical change
Conversation state	The messages and working fields carried through the graph.	Date and memory messages; summary; todos; title; replacement AI message.
Model request copy	The package prepared for this particular model call.	User boundaries, repaired tool history, receipt ledger, budget warning, PII masking.
Trace metadata	Labels that help you inspect the run in LangSmith.	dearflow_hook, langgraph_step and ls_run_depth. These are not automatically prompt text.

State hook

BEFORE

```
Before: current state
```

AFTER

```
After: a small state_update patch, or null. The graph merges the patch.
```

The patch is not a complete replacement transcript. A message with the same id can replace an earlier message.

Model wrapper

BEFORE

```
Before: current model request
```

AFTER

```
After: request passed to the next layer; later, the response returned from that layer.
```

A wrapper can edit this outgoing copy without rewriting the stored user message.

No change can be the correct result

Enabled does not mean triggered. Short history needs no summary; a healthy tool history needs no repair; a turn below the budget limit needs no warning.

Original guide: p. 27, p. 28, p. 29, p. 30, p. 35, p. 36, p. 37

before_agent

Prepare the conversation once, before the agent starts handling this user request.

Execution order
DynamicContextMiddleware -> MemoryMiddleware

Layer	Why it exists	Before -> after
DynamicContext	Help the model interpret dates such as today.	No current-date reminder -> a hidden date reminder, when needed.
Memory	Carry useful saved preferences into a fresh conversation.	No saved facts in this turn -> a hidden <memory> block, when accepted facts exist.

Illustration: fresh thread

BEFORE

```
User: What is my favourite colour?
```

AFTER

```
Hidden date reminder
Hidden memory: favourite colour is green
User: What is my favourite colour?
```

Shortened illustration, not a verbatim trace. Hidden means application-added context, not a new user instruction.

What repeats?

The next tool result does not restart before_agent. A new user message creates another invocation and can run these hooks again. A hook may return null if there is nothing to add.

Where to look in LangSmith

Open middleware.DynamicContextMiddleware.before_agent or middleware.MemoryMiddleware.before_agent. Compare inputs.before.value with outputs.state_update.value, then check the next hook input for the merged messages.

Original guide: p. 6, p. 16, p. 17, p. 18
LangSmith screenshots: p. 43, p. 44

04 / PREPARE THE NEXT MODEL STEP

before_model

Check the shared conversation state each time the loop is about to call the model.

Execution order
RedactingSummarizationMiddleware -> TodoMiddleware

Layer	Why it exists	Before -> after
Summarization	Keep a long conversation within a useful context size.	Older messages -> summary plus retained recent messages, only after its threshold is crossed.
Todo	Keep the plan available if earlier planning context disappeared.	Saved todos without their context -> a hidden reminder of the saved todo list, when needed.

Illustration: long conversation

BEFORE

```
Many older messages
Recent question and tool results
```

AFTER

```
Summary of older conversation
Retained recent messages
Optional reminder of the saved plan
```

Summarization returns a state patch that removes/replaces history. It can use a separate model call to generate its summary.

State preparation is not all prompt preparation

User-text boundaries, request-history repair and runtime PII masking happen later in wrap_model_call. A null before_model patch does not mean the final model input is unchanged.

Where to look in LangSmith

For summarization, inspect outputs.state_update.value for removal markers and the summary. For Todo, inspect the reminder patch. Follow the next wrapper to see the resulting request.

Original guide: p. 6, p. 18, p. 19, p. 23, p. 24
LangSmith screenshots: p. 48, p. 52

wrap_model_call

Wrappers nest around the same model call. The request travels inward; the generated reply travels outward in reverse order.

Execution order
ToolReceipt -> InputSanitization -> DanglingToolCall -> TokenBudget -> LoopDetection -> Todo -> PiiRedaction -> MODEL

Layer on entry	Possible request change
ToolReceipt	Add a ledger describing recorded tool outcomes.
InputSanitization	Add user-input boundary markers and escape reserved control tags.
DanglingToolCall	Fill missing tool responses and remove orphan tool responses in the request copy.
TokenBudget / LoopDetection	Add a warning when the corresponding threshold requires it.
Todo	Add system instructions for planning and completing work.
PiiRedaction	Mask supported personal data in eligible user text before the model sees it.

Illustration: plain user message

BEFORE

```
User: What is 17 times 23?
```

AFTER

```
--- BEGIN USER INPUT ---
What is 17 times 23?
--- END USER INPUT ---
```

This shows sanitization only. The later layers can make additional changes. These boundaries are not a guarantee that all prompt injection is prevented.

Return order: PiiRedaction -> Todo -> LoopDetection -> TokenBudget -> DanglingToolCall -> InputSanitization -> ToolReceipt. Most wrappers leave the reply text alone. ToolReceipt adds a ledger in the AI message bookkeeping.

Where to look in LangSmith

Entry comparison: inputs.before.value -> outputs.calls[0].request.value. Return comparison: outputs.calls[0].response.value -> outputs.result.value. The outer wrapper request is not necessarily the final provider-facing request.

Original guide: p. 6, p. 13, p. 14, p. 15, p. 21, p. 22, p. 23, p. 24, p. 25
LangSmith screenshots: p. 42, p. 45, p. 47, p. 55, p. 56

The model reads and replies.

The innermost model receives the prepared messages and system instructions, with available tools and model settings supplied by the application.

Illustration: a tool request

BEFORE

```
User asks for 17 times 23.
The model can use a calculator tool.
```

AFTER

```
AIMessage
content: optional explanation
tool_calls: [{name: calculator, ...}]
usage_metadata: token counts
```

Conceptual example, not literal tool argument JSON. A requested call is a proposed action; it has not run yet.

Output	What it tells you
content	Text generated by the model. It can be empty when the model only asks for tools.
tool_calls	Tool names, arguments and ids requested by the model. after_model can still remove or restrict them.
usage_metadata	Input, output and total token use for this model call, when the provider reports it.

Count model calls, not wrapper boxes

Seven surrounding wrapper spans can all contain the same model call. Adding parent and child totals double counts it. A user turn may also contain subagent calls and a summarization call. Background memory extraction can be outside the turn trace.

Where to look in LangSmith

Select the innermost ChatOpenAI LLM span. Input is the final provider-facing message list; tool schemas and settings can appear in invocation parameters. Output shows what the model generated before later graph guards. Do not treat this display as a complete raw network payload.

Original guide: p. 3, p. 4, p. 6, p. 31, p. 36, p. 39, p. 41

07 / CHECK THE GENERATED REPLY

after_model

Check the new AI message before acting on its requested tools or accepting its answer as complete.

Execution order
Clarification -> Todo -> LoopDetection -> TokenBudget -> SubagentLimit

Guard	Why it exists	Possible after state
Clarification	Avoid doing unrelated actions while asking the user a question.	Keep ask_clarification and remove sibling tool requests.
Todo	Avoid ending while the plan still contains unfinished tasks.	Add a reminder and jump_to=model, subject to its reminder limit.
LoopDetection	Stop repeated calls with the same tool and arguments.	Remove tool calls and add a stop explanation.
TokenBudget	Stop further tool work when the turn budget is exhausted.	Check recorded usage; remove further tool calls at the hard limit.
SubagentLimit	Bound delegation per reply and per turn.	Keep the allowed task calls; remove excess task calls and add a warning.

Illustration: delegation guard

BEFORE

```
AI requests three task calls.
```

AFTER

```
If only two are allowed: keep two task calls and add a warning.
```

The removed task request is not executed. This is a conditional example, not a claim about the configured limit in every run.

Order and rerouting matter

After hooks normally run in reverse registration order. Todo jump_to=model returns to the loop entry at before_model and can skip later after_model hooks in that iteration. A hard token stop is reactive: the reply just generated can already exceed the limit.

Where to look in LangSmith

Compare inputs.before.value with outputs.state_update.value. Check messages, tool_calls and jump_to. A replacement AI message often keeps the same id, so it replaces the previous version rather than adding another answer.

Original guide: p. 6, p. 20, p. 21, p. 22, p. 23, p. 24, p. 25, p. 26
LangSmith screenshots: p. 49, p. 50, p. 51, p. 52, p. 53

wrap_tool_call

Wrap each tool action that survives the response guards. A reply with several tool requests can produce several tool executions.

Execution order
ToolReceipt -> ToolErrorHandling -> PiiRedaction -> Clarification -> TOOL

Layer	What happens
Clarification	For ask_clarification, supply the formatted question directly without calling the inner handler. Other tools continue inward.
PiiRedaction	On a normal return, mask supported personal data from configured external tool outputs. It does not rewrite every tool result.
ToolErrorHandling	Catch ordinary tool exceptions and turn them into an error ToolMessage the model can understand.
ToolReceipt	Stamp the final result with execution bookkeeping so later model calls can see recorded outcomes.

Example: calculator failure

BEFORE

```
Permitted call asks the calculator to divide 1
by 0.
```

AFTER

```
ToolMessage with status=error
Readable failure information
Tool receipt bookkeeping
```

The tool failed, but the agent can read the error in its next model step and recover. The full guide includes a real calculator error trace.

Two exits
Ordinary tool result -> before_model -> another model call. Enabled ask_clarification uses return_direct -> after_agent -> wait for the user. Its handler_calls=0 is expected because the wrapper handled it.

On an exception path, the inner PII layer receives no normal result. ToolErrorHandling catches the exception outside it and creates the error result; ToolReceipt then stamps that result.

Where to look in LangSmith
Inspect inputs.before.value for the requested tool and arguments, then outputs.calls and outputs.result. Follow the actual tool result and receipt, not only the earlier AI tool request.

Original guide: p. 6, p. 13, p. 14, p. 16, p. 24, p. 25, p. 26, p. 38
LangSmith screenshots: p. 45, p. 46, p. 53

after_agent

Run completion work after the agent loop finishes normally. These hooks do not run after every individual model call.

Execution order
TitleMiddleware -> MemoryMiddleware

Layer	Why it exists	Before -> after
Title	Give the conversation a useful short label.	Missing title -> title state field, when required. This does not rewrite the final answer.
Memory	Learn useful facts from the completed exchange for a later turn.	Completed exchange -> queued extraction work. Usually no state patch.

Illustration: learning then recalling

BEFORE

```
Turn A: My favourite colour is green.
The agent replies, and after_agent runs.
```

AFTER

```
Later: extractor accepts a fact and storage
saves it.
Turn B, fresh thread: What is my favourite
colour?
before_agent injects the saved fact.
```

Simplified illustration. The real successful test used a green notebook preference and concise bullets. Saving and recalling are separate events.

A queued update is not a saved fact

Memory can return null and still queue work. A successful after_agent span proves the hook returned; it does not prove extraction succeeded. The guide also shows a historical invalid-JSON extraction failure.

Normal clarification completion also goes through after_agent. An unhandled error or aborted graph may prevent the normal completion path. In the recorded configuration, title generation uses a local fallback; enabling model-based titles would add a model call.

Where to look in LangSmith

Title: inspect outputs.state_update.value.title. Memory: verify the later extraction result and accepted saved facts, then a fresh-thread recall without supplying the answer. Do not use the assistant saying “I will remember” as proof.

Original guide: p. 5, p. 11, p. 12, p. 17, p. 18, p. 19, p. 20, p. 40, p. 60, p. 61
LangSmith screenshots: p. 44, p. 54, p. 59

A simple two-model-call turn.

Illustrative prompt: “Use the calculator to work out 17 times 23, then tell me the answer.” Assume no guard threshold is reached and no unfinished plan forces a retry.

Step	What happens	Visible outcome
1. before_agent	Date and saved facts can enter state.	Context is ready for this invocation.
2. before_model	Check history length and saved plan.	Often no change for a short task.
3. Wrappers -> model #1	Prepare request; model requests calculator.	AI tool request, not an executed result.
4. after_model	Guards inspect the request.	Calculator call remains allowed.
5. Tool wrappers -> calculator	Execute 17 times 23 and stamp result.	Tool returns 391 plus receipt metadata.
6. Back to before_model	Tool result is now in state; request wrappers repeat.	Receipt ledger can tell the model what ran.
7. Model #2 -> after_model	Model uses the result and answers.	Final answer: 391; no remaining tool calls.
8. after_agent	Set title if needed; queue memory processing.	This user turn completes.

This is a teaching example

It is not a newly executed test or an exact transcript. A real model may plan, ask a question or make a different number of calls. Guard demonstrations in the full guide used controlled fixtures or thresholds.

Real evidence: the full-flow retry in the guide made 11 LLM calls: 7 lead-agent calls plus 4 subagent calls. It exercised planning, two delegations, tool-error recovery and calculator work. Separate tests were needed for failure limits, summarization and memory. There is no single natural turn that should force every middleware to change data.

Original guide: p. 8, p. 9, p. 10, p. 11, p. 12, p. 60, p. 61

Where each component fits.

Use this index to jump from a lifecycle stage to the detailed explanation and real LangSmith screenshot in the original PDF.

Middleware	Hooks	Why it exists	Guide pp.	Screens pp.
ToolReceipt	WM, WT	Record tool outcomes	13, 14	45
InputSanitization	WM	Mark user-text boundaries	14	42, 57, 58
DanglingToolCall	WM	Repair tool-message pairing	15	47
ToolErrorHandling	WT	Return usable tool errors	16	46
DynamicContext	BA	Add the current date	16, 17	43
Memory	BA, AA	Recall and queue saved facts	17, 18	44, 59
RedactingSummarization	BM	Shorten older conversation	18, 19	48
Title	AA	Name the conversation	19, 20	54
SubagentLimit	AM	Limit delegation	20	49
TokenBudget	WM, AM	Track and limit tokens	21	51
LoopDetection	WM, AM	Stop repeated tool calls	22	50
Todo	BM, WM, AM	Keep plans on track	23, 24	52
PiiRedaction	WM, WT	Mask eligible personal data	24, 25	55, 56
Clarification	AM, WT	Ask and wait for the user	25, 26	53

Short hook names: BA = before_agent; BM = before_model; WM = wrap_model_call; AM = after_model; WT = wrap_tool_call; AA = after_agent. Entry order is registration order; after hooks and wrapper returns reverse it.

Original guide: p. 6, p. 7, p. 13, p. 14, p. 15, p. 16, p. 17, p. 18, p. 19, p. 20, p. 21, p. 22, p. 23, p. 24, p. 25, p. 26

Compare the right two objects.

Open the user-turn trace, expand its children, and select a custom middleware.<Class>.<hook> span. Match the comparison to the hook type.

Question	Compare these exact paths
What state did a before/after hook change?	inputs.before.value -> outputs.state_update.value
What request did a model wrapper pass inward?	inputs.before.value -> outputs.calls[0].request.value
What response did a model wrapper return outward?	outputs.calls[0].response.value -> outputs.result.value
What did a tool wrapper execute or intercept?	inputs.before.value -> outputs.calls / outputs.result
What messages reached the actual model?	Innermost ChatOpenAI span Input; check invocation parameters for tool definitions and settings.

Keyword	Simple meaning	Original PDF
dearflow_hook	Which hook this custom span records.	p. 29
dearflow_middleware	Which middleware component produced it.	p. 29
langgraph_step	Graph execution step; not your chat-turn number.	p. 30
ls_run_depth	Nesting depth in the trace; not number of model calls.	p. 31
handler_calls	How many times the wrapper invoked its next handler.	p. 35
redaction_applied	Snapshot-export masking flag; not proof of runtime PII transformation.	p. 35
jump_to	A routing instruction such as model; not prompt text.	p. 36

Use snapshots and the original glossary together

The complete metadata dictionary is on original pages 29-41. A null state_update can be a valid no-op. If a snapshot is shortened or export-masked, its visible content is limited evidence; follow the raw model span and the guide's caveats.

Original guide: p. 27, p. 28, p. 29, p. 30, p. 31, p. 35, p. 36, p. 37, p. 38, p. 39, p. 40, p. 41
LangSmith screenshots: p. 57, p. 58

Keep these boundaries in mind.

This map follows the lead agent in the simplified DearFlow implementation inspected on 5 October 2026. Configuration and exceptional routing can change which hooks run.

Common assumption	Accurate reading
Every middleware changes the model input.	Some change state, a tool result, routing or a title. Some only act when a trigger is present.
after_model is every operation after the provider replies.	Wrapper return processing happens first. after_model is a separate graph hook after the wrapped call returns.
Every subagent uses all 14 components.	Subagents use a smaller chain: Receipt, ToolErrorHandling, InputSanitization, DanglingToolCall, TokenBudget, LoopDetection and PII.
The user reply to a clarification is the next model step in the same run.	It starts a new invocation on the same conversation thread.
Runtime PII masking protects every stored and generated message.	Its scope is eligible outgoing user text and configured external tool outputs. It is not general final-answer redaction or global storage redaction.
A successful memory hook means persistence succeeded.	It may only have queued work. Inspect later extraction, saved facts and fresh-thread recall.

Async hook names

Names such as abefore_agent or awrap_model_call are asynchronous versions of the same lifecycle stages, not additional stages.

Implementation references: src/dearflow/middlewares/chain.py defines registration; agents/lead_agent.py wires the chain into create_agent; the installed LangChain factory supplies hook ordering and routing. Exact source paths and line references are included in references.json beside this guide.

This companion reuses the existing guide and verified source. It does not rerun models, change middleware configuration or create new LangSmith evidence. Keep both report folders together so the relative PDF page links continue to work.

Original guide: p. 6, p. 7, p. 27, p. 28, p. 60, p. 61, p. 62